

PROJECT PIPELINE

NeuroTrace-DAG

Recommend the right learning resource, to the right student, at the right time — accounting for what they have probably forgotten and what they are ready to learn next.

Status as of 26 September 2026. Working data, embeddings, and the prerequisite graph are in English.

1. What this system is

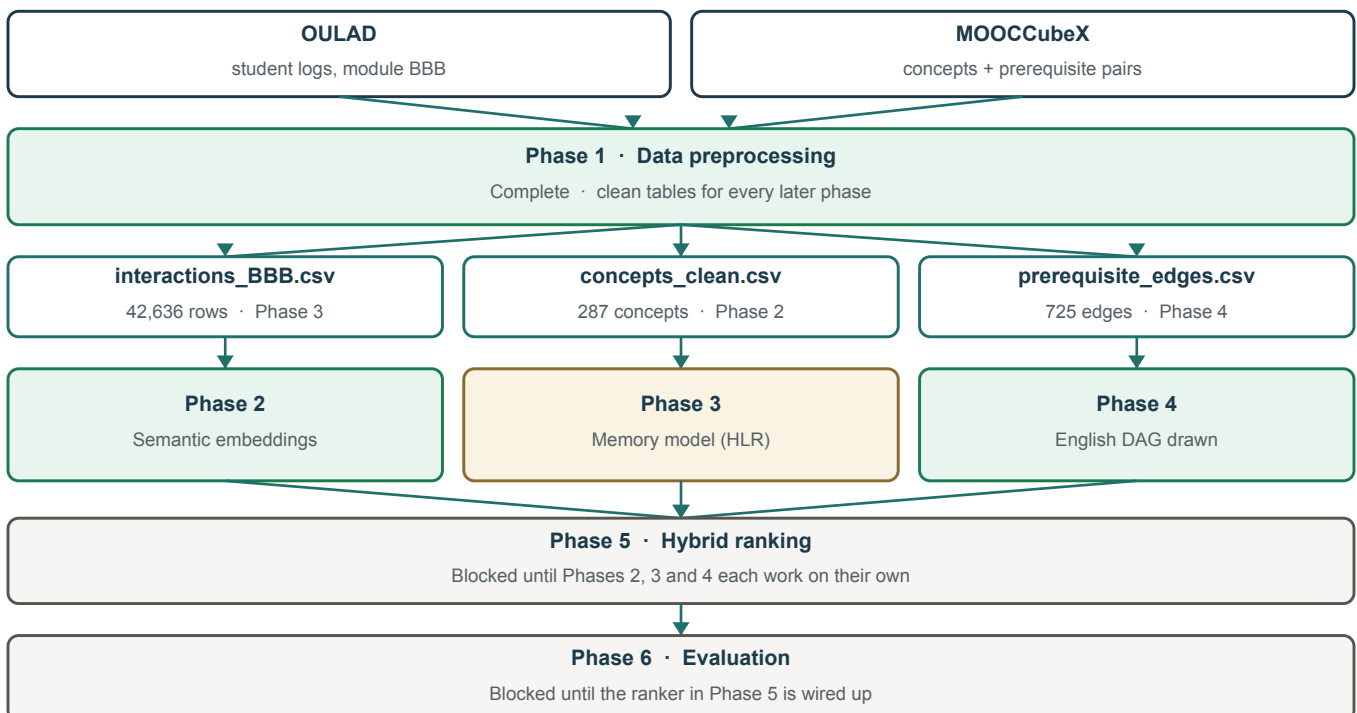
NeuroTrace-DAG is a pipeline of four components, combined at the end into one ranked list. Only one of them is a model we train.

Component	Training?	Role
Semantic embeddings	No. A pretrained model is used as-is.	Phase 2. How similar two texts are.
Memory retention (HLR)	Yes. The one trained model.	Phase 3. Probability the student still remembers.
Prerequisite DAG	No. Built from confirmed edges.	Phase 4. Whether the student is allowed to move on.
Ranking	No in this version. Weights can be tuned later.	Phase 5. One score, then the top of the list.

Phases 2, 3 and 4 do not depend on each other. They can be built in parallel. Phase 5 waits until all three produce their outputs. Phase 6 waits on Phase 5.

2. The pipeline

Raw OULAD and MOOCCubeX enter Phase 1. Three clean tables come out. Each table feeds one middle phase. Those three phases meet in the ranker.



Green is finished. Amber can start now. Grey is blocked on earlier phases. Teal boxes are the files that carry data from one phase to the next.

Phase	Status	Reads	Writes
1. Data preprocessing	Complete	Raw OULAD + MOOCCubeX	Three clean tables
2. Semantic embeddings	Complete	concepts_clean.csv	concept_embeddings.npy
3. Memory model (HLR)	Can start	interactions_BBB.csv	A function p(t)
4. Prerequisite DAG	Graph drawn	prerequisite_edges.csv	prerequisite_dag.pdf. Unlock rule still open.
5. Hybrid ranking	Blocked	Outputs of 2, 3 and 4	Top-K recommendations
6. Evaluation	Blocked	A working ranker	Metrics and ablations

3. Phase 1 — Data preprocessing

Status: complete. This phase turns two differently structured datasets into tables the later phases can read.

3.1 OULAD

OULAD records who did what, and when, inside a real online course. Only module BBB was processed, from studentInfo, studentAssessment, studentVle and assessments. For every assessment attempt the script computed two fields:

- `delta_t` — days since that student's last click on the course site before the assessment. This is the time gap the memory model will use.
- `recall_label` — 1 if the score was at least 40, otherwise 0. This is the pass/fail target for Phase 3.

Output: **interactions_BBB.csv**, 42,636 rows. Phase 2 does not read this file. Phase 3 does.

3.2 MOOCCubeX

MOOCCubeX records what a topic means and which topic must come first. `concept.json` (637,572 concepts) and `cs.json` (492,102 candidate pairs) were downloaded from the public links and parsed as JSON Lines, one object per line. Pairs were kept only when a human had marked them as a real prerequisite (`ground_truth = 1`). That leaves 725 edges and 287 concepts. Duplicate concept entries, one per academic domain, were resolved by preferring the Computer Science tag and the richest context text.

Each concept received a **text_for_embedding** field: the joined context when it exists, or the bare concept name when it does not. 214 concepts have real context. 73 fall back to the name alone. The source text was Chinese. The working tables are now English: concept names are checked computer-science terms, and the descriptions were translated. A scan of `concepts_clean.csv` and `prerequisite_edges.csv` found no Chinese characters.

3.3 Files Phase 1 handed on

File	Rows	Used by
<code>interactions_BBB.csv</code>	42,636	Phase 3 only
<code>concepts_clean.csv</code>	287	Phase 2. Columns: <code>id</code> , <code>name</code> , <code>domain</code> , <code>context_len</code> , <code>text_for_embedding</code>
<code>prerequisite_edges.csv</code>	725	Phase 4. Columns: <code>source_concept_name</code> , <code>target_concept_name</code>
<code>concepts_clean_zh.csv</code>	287	Chinese backup only. Do not feed it to a later phase.
<code>prerequisite_edges_zh.csv</code>	725	Chinese backup of the edges. Do not feed it to a later phase.

The two datasets are not linked by an ID. `interactions_BBB.csv` describes one whole module. The 287 concepts are fine-grained topics, now named in English. Joining a student to those concepts is still a Phase 4 mapping, because one side is a whole module and the other is 287 separate topics.

4. Phase 2 — Semantic embeddings

Status: complete. No training. The encoder is used exactly as published.

Keyword match cannot tell that two different phrases are about the same idea. Phase 2 turns each concept's text into a vector of 384 numbers so relevance can be measured by how close those vectors are. The input is the `text_for_embedding` column of `concepts_clean.csv`, all 287 rows.

The working concept text is English. The encoder is **paraphrase-multilingual-MiniLM-L12-v2**, used as published. It embeds this English text. The matrix was rebuilt after the tables were translated, so every vector matches the English row it sits on.

What the script does:

- Load `concepts_clean.csv` and refuse to run if `text_for_embedding` has empty cells.
- Encode every text with the pretrained model. Vectors are L2-normalised, so cosine similarity is a dot product.
- Save one vector per concept in `concept_embeddings.npy`. Row *i* is the concept on row *i* of `concepts_clean.csv`. Shape (287, 384), float32.
- Check neighbours on 10 concepts: 5 with real context and 5 with the bare name only. Also compare all 725 prerequisite pairs with 725 random pairs.

At 287 concepts a full comparison is cheap. No approximate search index is used. The model keeps the first 128 tokens of each text, so a long encyclopedia snippet is only partly read. The concept name sits at the start of the text, so that part is kept.

4.1 Sanity check

Prerequisite-linked pairs are closer than random pairs of the same count. Mean cosine is 0.292 for linked pairs and 0.194 for random pairs. The gap holds inside the context-rich group (0.291 versus 0.207) and inside the name-only group (0.343 versus 0.215). Name-only vectors sit closer together in general, so those 73 embeddings are less specific.

Probe	Context?	Nearest concepts, in order
interrupt controller	Yes	interrupt signal, interrupt control, interrupt mechanism
storage allocation	Yes	dynamic memory allocation, segmented memory management, auxiliary memory
B-tree	Yes	leaf node (its only graph neighbour, rank 1)
paging	Yes	logical address space, auxiliary memory, page fault
interrupt service routine	Name only	interrupt enable, interrupt control, enable interrupts
processor scheduling	Name only	multiprocessor scheduling (cosine 0.85)
Internet Protocol	Name only	Routing Information Protocol, connection-oriented protocol

Full top-5 lists are in `phase2_sanity_neighbors.csv`. The numeric summary is in `phase2_sanity_summary.txt`.

A graph neighbour and a semantic neighbour are related ideas, and they are not the same relation. Paging's only confirmed prerequisite is interrupt mechanism, which did not appear in its top 5. The nearest vectors were still about memory: logical address space, auxiliary memory, and page fault. The embedding is doing the job Phase 2 asked of it.

4.2 Files Phase 2 handed on

File	What it is
phase2_semantic_embeddings.py	Encodes the 287 texts and writes the sanity check.
concept_embeddings.npy	The matrix later phases load. Row order matches concepts_clean.csv.
phase2_sanity_neighbors.csv	Top-5 neighbours for the 10 probe concepts.
phase2_sanity_summary.txt	Pair scores, split by context, and the written limitations.

Phase 5 should load `concept_embeddings.npy` and compare a query vector with those rows by dot product. It should not re-encode the catalog unless the concept table changes.

5. Working language

The pipeline now runs in English. `concepts_clean.csv`, `prerequisite_edges.csv`, the embedding matrix, and the drawn graph all use English concept names.

MOOCCubeX arrived in Chinese. Concept names were set to checked computer-science terms, such as paging, virtual page, file allocation, and interrupt controller. Descriptions were translated into English, then scanned: the working tables contain no Chinese characters. `concepts_clean_zh.csv` and `prerequisite_edges_zh.csv` keep the Chinese originals as backups. Later phases do not read those backups. `interactions_BBB.csv` was already English.

Phase 2 was run again after the translation. Row *i* of `concept_embeddings.npy` is still row *i* of `concepts_clean.csv`, now from the English text.

6. Phase 3 — Memory retention

Status: not started. This is the only phase that trains a model. It can start now. It does not wait on Phase 2 or Phase 4.

Given a student and a topic they studied before, predict the probability they still remember it. The input is `interactions_BBB.csv`. The curve is the half-life form from Settles and Meeder (2016):

$$p(t) = 2^{(-\Delta t / h)}$$

$p(t)$ is the probability of recall. Δt is `delta_t`, in days. h is the half-life: the number of days until recall probability falls to one half. The model learns h .

$$h = 2^{(w \cdot x)}$$

x is a feature vector built from columns the interactions file actually supports, such as `delta_t`, past attempts and past success. w is the weight vector. Training compares predicted $p(t)$ with the observed recall label, using binary cross-entropy or the squared error from the original paper. The split is by student, about 80/20, so the test students were never seen in training. A first pass can use scikit-learn. A small PyTorch loop can match the paper's formula more closely. Report MAE and AUC on the held-out students.

Label imbalance, to decide before training

At a pass threshold of 40, `recall_label` is about 96.7% pass and 3.3% fail. A model that always says pass would look accurate and would have learned nothing. AUC is unstable with so few failures. The Phase 3 owner picks one of these and writes the choice down:

- Raise the pass threshold and state the new value.
- Use the raw score, scaled to the range 0–1, as a continuous target. This matches a probability and avoids the split into pass and fail.
- Keep the binary label, weight the classes, and report AUC and precision-recall AUC rather than accuracy.

Output: a function that takes a student, a concept and the current time, and returns $p(t)$.

7. Phase 4 — Prerequisite graph

Status: the English graph is drawn and it has no cycles. The unlock rule and the OULAD mapping are still open.

- The directed graph is already built from the English edges. An edge runs from `source_concept_name` to `target_concept_name`: learn the source before the target. The drawing is `prerequisite_dag.pdf` (9 pages) and `prerequisite_dag.png`.
- The graph has no cycles: 287 concepts, 725 edges, longest path 26 edges.
- A concept is unlocked when every predecessor has mastery above a threshold (a working value is 0.70, taken from the Phase 3 recall probability).
- Expose `is_unlocked(student, concept)`, returning true or false.

Linking OULAD to MOOCCubeX

Student mastery comes from OULAD. The graph comes from MOOCCubeX. A full automatic join is not realistic: there are no shared IDs, and one side is a whole module while the other is 287 fine-grained concepts. Both sides are now in English. The plan is a small, manually curated mapping from module BBB's topics to a subset of those 287 concepts. Each row of that table should say how the mapping was chosen. Reviewers will ask. The graph will constrain only the mapped subset. Many concepts have few or no prerequisites, because 725 edges on 287 concepts is a benchmark subset, so those concepts will look unlocked immediately.

8. Phase 5 — Hybrid ranking

Status: blocked. Start only after Phases 2, 3 and 4 each work on their own.

For one student and one query, produce a ranked list.

- Candidates: concepts whose vectors are close to the query (Phase 2).
- Filter: drop any candidate that is not unlocked, meaning a prerequisite is still below the mastery threshold (Phase 4).
- Score the rest and return the top of the list.

$$\text{Score}(r) = w_1 \cdot \text{SemanticRelevance}(\text{query}, r) + w_2 \cdot (1 - p(t)) + w_3 \cdot \text{CurriculumProgress}(r)$$

`SemanticRelevance` is the cosine from Phase 2. $(1 - p(t))$ is larger when the memory model thinks the concept has been forgotten. `CurriculumProgress` is a measure of how far the concept sits along the graph relative to what the student has already mastered, for example normalised depth from the student's frontier. Start the three weights equal, at one third each. Hand-set weights are the weakest part of this design. Once the pipeline runs, search them against a validation metric, or record them as a limitation.

9. Phase 6 — Evaluation

Status: blocked. It measures a pipeline that already runs.

Metric	What it measures
MAE	How far HLR's predicted recall is from the held-out outcome.
AUC	How well HLR separates "will recall" from "will not". Read it next to the class balance from Phase 3.
NDCG@K	Whether the ranked list puts relevant concepts first. This needs relevance labels, which the datasets do not provide directly.
Prerequisite violation rate	The share of recommendations whose prerequisites were not unlocked.
Inference latency	End-to-end time for one recommendation, averaged over many queries.

Ablations, so each component has to earn its place:

- Semantic only: rank by cosine, with no memory term and no graph filter.
- No DAG: the full score, with the prerequisite filter removed.
- No memory: the full pipeline with the forgetting weight set to zero.

State the scope in the write-up: results cover OULAD module BBB only, the graph applies only to the manually mapped concepts, and NDCG needs labels that must be constructed.

10. Build order

- Phase 1 is done. The three tables are the contract for everything after it.
- Phase 2 is done. `concept_embeddings.npy` is the contract for semantic relevance.
- Phase 3 can start from `interactions_BBB.csv`. The English DAG is already drawn; Phase 4 still needs `is_unlocked()` and the manual mapping from module BBB.
- Wire Phases 2, 3 and 4 into the Phase 5 score.
- Run Phase 6, including the three ablations.

11. Limitations to carry into the report

- The working concept text is English. Names are checked terms. Long descriptions are translations of noisy encyclopedia snippets, so a sentence can still be awkward even though no Chinese remains.
- 73 of 287 concepts have no context text. Their embeddings are weaker and less specific. The Phase 2 numbers above are split so this can be seen.
- The model reads 128 tokens. Long context is only partly used. Neighbours were still on topic.
- The prerequisite graph is a sparse benchmark subset: 725 edges on 287 concepts, not a full curriculum.
- OULAD and the concept graph still meet only through a manual mapping, which Phase 4 still has to write. Language is no longer the obstacle.
- `recall_label` is about 96.7% pass. How to handle that is a Phase 3 decision, still open.
- Only OULAD module BBB is used.
- Ranking weights are hand-set until Phase 5 has a validation metric to tune them against.

12. Tools

Step	Library
Preprocessing	pandas, numpy
Embeddings	sentence-transformers, model paraphrase-multilingual-MiniLM-L12-v2

Step	Library
Similarity	numpy dot product on normalised vectors
HLR training	scikit-learn for a first pass; PyTorch if the paper's formula is coded directly
Graph	networkx and Graphviz. English drawing: prerequisite_dag.pdf
Evaluation	scikit-learn.metrics
Hardware	CPU. No GPU is required at this scale.

Scripts in the project: `phase1_oulad_preprocessing.py`, `phase1_mooccubex_preprocessing.py`, `phase2_semantic_embeddings.py`.
Re-running Phase 2 rewrites `concept_embeddings.npy` and the two sanity files. It does not change the Phase 1 tables.